Urios Matthieu
Bonhoure Hector

Rapport du projet PFA

I - Présentation du jeu et manuel d'utilisation **2**
1/ Présentation du jeu 2
2/ Manuel d'utilisation 2
Commandes de bases 2
Les armes à disposition 2
Les différents ennemis 3
La mini-map 3
II - Architecture du code et types principaux **4**
Architecture du code 4
Présentation des systèmes 5
III - Présentation de points techniques du code **6**
1/ Premier point : gestion des armes et des interactions 6
Présentation du problème 6
Les intéractions 6
Les armes 8
2/ Deuxième point : gestion du donjon 9
Présentation du problème 9
Structure de donnée 9
Algorithme de génération en quatre étapes 11
Changement de salle 11
Bilan 12
IV - Tests effectués **12**
Approche générale 12
Tests par affichage de debug 12
Tests fonctionnels ciblés 13
Problèmes connus non corrigés 13
V - Répartition du travail **14**
Travail effectué ensemble 14
Travail effectué par Matthieu Urios 14
Travail effectué par Hector Bonhoure 14

I - Présentation du jeu et manuel d’utilisation

1/ Présentation du jeu

Dans “Neon PFA” vous évoluez dans une ville à l’aspect futuriste avec un seul but : **Réussir la casse du siècle, c'est-à-dire voler la plus précieuse couronne du fameux musée du L0UVR3**. Cependant la tâche ne sera pas aisée, votre chemin est parsemé d'embûches. Mais il n’y pas de problèmes de ce genre qui ne se règlent pas à bon coup de flingues !
Vous serez accompagné par 4 différents alliés, votre fidèle glock, un vieux fusil à pompe qui fait encore des merveilles, un lance roquette comme on en fait plus et un fusil laser à la pointe de la technologie. Arriverez vous à traverser la ville en restant en un seul morceau ? Nous vous souhaitons bonne chance, vous en aurez besoin…

2/ Manuel d’utilisation

Commandes de bases

Notre jeu requiert un clavier et une souris pour une expérience optimale.

Pour les déplacements il faut utiliser les touches **Z**, **Q**, **S** et **D**.
Pour faire feu avec votre arme, il faut amener le curseur de la souris à l’endroit où vous désirez tirer puis faire un **clic gauche**.
Pour interagir que ce soit avec une arme au sol ou une porte pour changer de salle, il vous faudra vous positionner sur l’entité en question et appuyer sur **E**.

C’est tout pour les commandes, passons maintenant à la description des armes qui vont vous accompagner dans votre aventure.

Les armes à disposition

Nom de l’arme : **Glock - P 80**
Votre plus ancien compagnon. Il a toujours été à vos côtés et ne vous à jamais fait défaut ; et ce n’est pas aujourd’hui que ça va commencer.
Dégâts par balle : **20**
Balles tirées par coup : **1**
Cadence de tir : **Rapide**

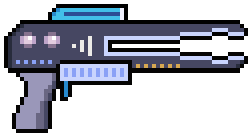
Nom de l’arme : **The Shotgun**
C’est une relique des temps anciens et il est toujours aussi efficace. Certaines choses restent toujour aussi utiles peu importe l’époque et cette arme en fait partie
Dégâts par balle : **20**
Balles tirées par coup : **5**

Cadence de tir : **Lente**

Nom de l'arme : **Bazooka - M 20**

Ah les explosions… Qu'il y a t'il de plus satisfaisant que réduire ses ennemis en poussière en un instant ? En tout cas, cette arme qui ne fait pas dans la dentelle allie efficacité et amusement pour en faire un feu d'artifice de pièces mécaniques.
Dégâts par roquettes : **200**
Roquettes tirées tirées par coup : **1**
Cadence de tir : **Lente**

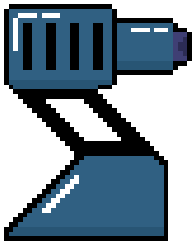
Nom de l'arme : **Laser - L 3000**



Arrachée à un garde automatique de la ville, cette arme à la pointe de la technologie est inarrêtable ! Déchaînez un rayon surpuissant sur vos ennemis et faites les fondre devant votre grandeur.
Dégâts par balle : **10**
Balles tirées par coup : **1**
Cadence de tir : **Vous n'avez jamais vu une cadence de tir aussi haute croyez moi…**

Les différents ennemis

Nom de l'ennemi : **Defence turret MK 3**



Les tourelles en charge de défendre la ville. Elles ne semblent pas aimer vous voir tout détruire sur votre chemin…
Points de vie : **100**
Dégâts par balle : **20**
Balles tirées par coup : **1**
Cadence de tir : **Moyenne**

Nom de l'ennemi : **Gregory**

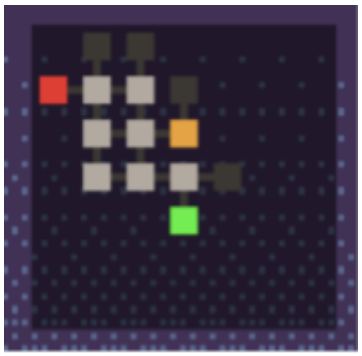
Son nom fait trembler même les plus grands. Vous avez pénétré sur son territoire et il ne compte pas vous laisser passer sans se battre !
Points de vie : **1000**
Dégâts par balle : **Il ne tire pas lui, il vous fonce dessus**
Balles tirées par coup : **0**
Cadence de tir : **Nulle**

La mini-map

Une mini-map est affichée en permanence en haut à droite de l'écran, permettant au joueur de se repérer dans le donjon sans en révéler l'intégralité. Elle repose sur un système de découverte progressive : une salle n'apparaît sur la mini-map que lorsque le joueur s'en approche ou la visite.

Les salles sont affichées avec un code couleur reflétant leur état d'exploration :

- **Blanc** : la salle a été visitée par le joueur.
- **Gris** : la salle a été découverte, c'est-à-dire qu'elle est adjacente à une salle visitée, mais le joueur ne s'y est pas encore rendu.
- **Orange** : la salle dans laquelle le joueur se trouve actuellement.
- **Vert** : la salle de départ.
- **Rouge** : la salle du boss.



Exemple de mini-map

Les connexions entre les salles sont également représentées sous forme de liens entre les cases de la mini-map, ce qui vous permettra de visualiser d'un coup d'œil la structure du donjon et d'anticiper les embranchements disponibles depuis sa position actuelle.
Cette information de découverte se retrouve également au niveau des portes : une porte menant vers une salle déjà visitée est affichée allumée, tandis qu'une porte menant vers une salle non encore visitée est affichée éteinte.

Salle non visitée

Salle visitée

II - Architecture du code et types principaux

Architecture du code

Nous avons lors du développement conservé l'architecture des TPs dans son ensemble. Les fichiers de codes sont séparés en trois dossiers, un dans lequel on trouve les différents composants comme les armes, le joueur, les ennemis, un dans lequel on trouve tous les différents systèmes et un dans lequel se trouve le coeur de notre jeu, c'est à dire des fichiers utilisées à la fois dans les systèmes et les composants.
Dans la suite nous allons donner une présentation rapide des différents composants et des systèmes de notre projet.
Le composant que nous utilisons le plus est block. En effet, il contient toutes les informations importantes pour gérer notamment l'affichage et les collisions. Ce composant n'a pas été beaucoup modifié par rapport au TPs si ce n'est qu'un attribut *tokill* lui a été ajouté. Lorsque cet attribut passe à true, le composant est automatiquement supprimé de tous les systèmes et supprimé du jeu.
Passons maintenant au composant camera. Elle dispose entre autres d'une vélocité et d'une position pour lui permettre de suivre le joueur dans ses déplacements. La position des différents objets sur l'écran se fait en fonction de la position de la caméra.
Puis vient le composant timer. Il est très important dans notre code car il nous permet de faire en sorte que certaines actions s'exécutent après un certain temps sans pour autant avoir à

mettre en pause l'entièreté du jeu. Lors de la création d'un timer on lui donne une durée et ce sera le timer_system, que nous détaillerons plus tard, qui va s'occuper de lancer l'action voulue à la fin de ce timer.
Passons à présent à tout ce qui est relatif aux armes. Le composant ammunition est juste un block avec un nom différent pour plus de clarté dans le code et représente les balles tirées par nos armes. Ces dernières sont représentées par un composant gun dans lequel est stocké entre autre l'ID de l'arme en question, sa cadence de tir, si on peut tirer avec ou non, la fonction qui va résoudre l'interaction entre une arme et un joueur et la fonction qui va être appelée lorsque l'on va tirer avec l'arme.
Les ennemis sont représentés par le composant enemy. Ils ont un attribut life qui stocke leurs points de vie actuels, un attribut action qui est l'action qu'ils vont effectuer tous les certains temps (comme tirer sur le joueur) et un dernier attribut can_drop_loot. Ce dernier sert à corriger un bug qui faisait que lorsqu'un ennemi se faisait toucher par plusieurs projectiles lors de la frame de sa mort, la fonction qui le fait lâcher des armes est appelée plusieurs fois ce qui le fait lâcher plusieurs armes. Ainsi lors de l'appel de cette fonction, on vérifie si l'attribut et a true auquel cas on lâche peut-être une arme et on le passe à false.
Passons au joueur représenté par le composant player. Ce composant stocke les informations sur l'arme actuelle du joueur ainsi que sa vie et si jamais il peut bouger.
Enfin il reste tout les trois composant qui serve au donjon : le composant door qui représente une porte que l'on peut emprunter pour changer de salle, le composant room qui représente une salle du donjon et enfin le composant dungeon qui représente le donjon et son agencement. Ces composant sont expliqués plus en détail dans la partie sur la gestion du donjon.

Présentation des systèmes

À présent, disons un mot rapide sur les systèmes.

- Le clear_system nous permet de supprimer du jeu et donc de décharger les éléments dont nous n'avons plus besoin.
- Le click_system va gérer tout ce qui est interaction avec les boutons de l'écran de victoire et de défaite.
- Le collision_system gère les collisions entre les différents éléments.
- Les systèmes draw_background, draw_death_menu, draw_lifebar, draw_map, draw_victory_menu et draw ont tous le même but, afficher certaines parties du jeu. Respectivement ils affichent, l'arrière plan du jeu (décors global + éléments qui ne doivent pas se retrouver affichés devant le joueur tels les portes), le menu de défaite, la barre de vie du joueur et des ennemis, la carte du donjon, le menu de victoire et enfin tout le reste des éléments (les murs, le sol, le plafond, les plateformes, les ennemis, les armes et le joueur).
- L'enemy_manager_system qui a chaque mise à jour vérifie si les ennemies peuvent agir et si jamais c'est le cas les fait agir.
- L'interact_system (détaillé plus précisément plus bas) qui gère les intéractions entre les différentes entités et le joueur.
- Le move_system qui gère les mouvements de tous les composant qui doivent se mouvoir

- Le player_manager_system sert à faire les modifications nécessaire sur le joueur (en jeu) en fonction des différentes touches sur lesquelles appuie le joueur (dans la vrai vie)
- Le room_loader_system s'occupe comme son nom l'indique de tout ce qui est relatif au chargement et au déchargement des salles lorsque le joueur bouge dans le donjon.
- Le timer_system qui va gérer les timer en cours. À chaque mise à jour il va enlever le dt depuis son dernier appel à tous les timer en cours et si jamais le temps qu'il leur reste est négatif ou nul il exécute l'instruction de fin du timer avant de le supprimer

III - Présentation de points techniques du code

1/ Premier point : gestion des armes et des interactions

Présentation du problème

Afin d'apporter de la diversité lors de la jouabilité de notre jeu, il nous a fallu implémenter différentes armes. Et qui dit possibilité de jouer plusieurs armes, dit qu'il nous faut pouvoir les interchanger. Dans notre jeu, les ennemis ont une chance de lâcher une arme sur le sol lorsqu'ils sont éliminés. Il a donc fallu que nous implémentions un système nous permettant dans un premier temps d'interagir avec ces armes. C'est ensuite avec l'implémentation du donjon, que nous détaillerons dans le prochain point, que ce système d'interaction s'est aussi avéré utile pour voyager entre les différentes salles.
Après avoir permis au joueur de changer d'armes il a aussi fallu lui permettre de tirer avec les armes en question dans la suite de ce point nous détaillerons comment nous parvenu à le faire.

Les intéractions

Lors de nos réflexions portant sur l'implémentation des intéractions, nous voulions qu'elles soient le plus intuitives possibles. Pour le joueur cela veut dire qu'il n'a qu'à se placer sur l'entité avec laquelle il veut interagir et appuyer sur la touche **E**.
Lors de l'appui de cette touche, la référence à une variable *is_interacting* situé dans le fichier **global.ml** passe à true puis à false lorsque la touche est relachée

```
let handle_interactions dt (e:t) =
  Global.is_interacting := Input.is_just_pressed "e"
```

Chaque élément avec lequel nous pouvons interagir dispose d'un attribut nommé *interact_resolver* qui est une fonction de type *unit -> unit* qui va être appelée pour résoudre l'interaction dont il est question. Voici un exemple de fonction qui résout ici l'interaction qui permet de ramasser une arme et de lâcher celle que l'on possède.

```
let interact_resolver (gun : gun) =
  (*Message de debug pour tester que l'interaction à bien lieu*)
  Gfx.debug "interaction avec gun d'id %d\n%!" gun#gun_id#get;
  let Global.{player1; _} = Global.get () in
  (*On vérifie si le joueur à une arme avant de la remplacer*)
```

```
  (match player1#curent_gun#get with
  (*On vérifie dans quelle direction regarde le joueur pour pouvoir déposer son arme actuelle
devant lui*)
  | Some g -> (if !Global.is_facing_left then
                let dim = g#box#get in
                g#position#set (Vector.add player1#position#get Vector.{ x = -1. -. (float
dim.width) ; y = 0.})
              else
                g#position#set (Vector.add player1#position#get Vector.{ x = (float
Cst.player_width) +. 1. ; y = 0.}));
   (*On réenregistre son arme actuelle dans tous les systèmes auquelle une arme au sol
appartient
   pour pouvoir l'afficher convenablement. On l'ajoute en plus dans la salle actuelle*)
   g#tokill#set false;
   Draw_system.(register (g :> t));
   Collision_system.(register (g :> t));
   Clear_system.(register (g :> t));
   Move_system.(register (g :> t));
   Interact_system.(register (g :> t));
   Room_loader.add_current_room (g :> deletable)
  (*Le joueur à toujour au minimum une arme donc on ne fait rien de ce cas*)
  | _ -> ());
  (*On affecte au joueur sa nouvelle arme et on la supprime de tous les systèmes et de la
salle.
  A présent elle n'est que stockée dans les données du joueur*)
  player1#curent_gun#set (Some(gun));
  gun#tokill#set true;
  Room_loader.remove_current_room (gun :> deletable)
```

Afin de savoir quand appeler ce type de fonction nous utilisons un *interact_system* dans lequel on enregistre toutes les entités avec lesquelles nous pouvons interagir. Décrivons rapidement son fonctionnement. Lors de la mise à jour de ce système nous allons vérifier si le joueur interagit. Si ce n’est pas le cas, on ne fait rien. Si jamais une interaction a lieu nous cherchons si une entité avec laquelle on peut interagir est superposée avec le joueur et si c’est le cas on appelle sa fonction de résolution d'interaction. Nous donnons le code ci-dessous.

```
let update _ el =
  let Global.{player1; _} = Global.get () in
  let playerbox = player1#box#get in
  let playerpos = player1#position#get in
  if !Global.is_interacting then begin
    let e_opt = Seq.find (fun (e:t) ->
      let p, r = Rect.mdiff playerpos playerbox e#position#get e#box#get in
      Rect.has_origin p r
      ) el
    in
    match e_opt with
    | None -> ()
    | Some e -> e#interact_resolver#get ()
  end
```

Avec cela nous en avons fini avec les intéractions. Détaillons maintenant comment nous faisons en sorte que le joueur tire avec les différentes armes qu'il a pu acquérir.

Les armes

Lorsque le joueur appuie sur la touche pour tirer nous appelons une fonction *handle_fire* donc voici le code

```
let handle_fire (gun : gun) =
  if gun#can_shoot#get then begin
    ignore(fire_func_ar.(gun#gun_id#get) ());
    gun#can_shoot#set false;
    Time.create_timer_no_room_clear
      gun#fire_rate#get
      (fun dt -> gun#can_shoot#set true);
  end
  else
    ()
```

Cette fonction va regarder quelle arme a le joueur et appeler la fonction qui crée la balle correspondante. Elle va ensuite modifier l'attribut *can_shoot* de l'arme pour le passer à false et lancer un timer d'une durée égale à la cadence de tir de l'arme au bout duquel cet attribut va passer à true. Pour plus de clarté nous donnons une fonction de création de balles ci-dessous, en l'occurrence celle pour le glock.

```
let fire_glock () =
  let a = new ammunition () in
  let Global.{main_camera ; player1 ; _} = Global.get () in
  (*Calculs des positions de départ et d'arrivé de la balle*)
  let x = player1#position#get.x +. float(Cst.player_width) /. 2. in
  let y = player1#position#get.y +. float(Cst.player_height) /. 2. in
  let target_x = ((float !Global.mouse_x) -. x) +. main_camera#position#get.x in
  let target_y = ((float !Global.mouse_y) -. y) +. main_camera#position#get.y in
  (*Définition des attributs de la balle*)
  let values = Block.{ Block.default_set_values with
    pos_x = x;
    pos_y = y;
    velocity = Vector.(mult 8. (normalize {x = target_x ; y = target_y}));
    texture = if target_x < 0. then !(Texture.glock_bullet_txt_reverted) else
!(Texture.glock_bullet_txt);
    width = 10;
    height = 10;
    resolve = (fun (v:Vector.t) (reacter:tag) -> resolve v a reacter);
    elasticity = 0.;
    tag = Ally_projectile_tag {damage = 20.}
    } in
  Block.set_block a values;
  (*Enregistrement de la balle dans la salle*)
  Room_loader.add_current_room (a :> deletable)
```

Voilà avec cela nous avons fini de détailler comment nous gérons les armes de l'interaction avec le joueur lors de leur ramassage jusqu'à la manière dont nous les faisons tirer. Passons donc au point suivant, la gestion du donjon.

2/ Deuxième point : gestion du donjon

Présentation du problème

L'un des points majeurs du projet était de générer procéduralement à chaque partie un donjon pour donner de la rejouabilité dans le jeu.
Ici nous avons décidé de modéliser le donjon par une grille de salles avec des portes qui les relie entre elles, une salle peut donc avoir une porte vers le haut, une vers la gauche, une vers le bas et une vers la droite qui renvoie dans la direction de la porte.
Ce problème a plusieurs enjeux, le premier étant de savoir quelle structure de données nous allons utiliser pour stocker le donjon et ses salles. Le deuxième est de savoir comment l'on va générer le donjon et le troisième étant de savoir comment l'on va pouvoir faire changer le joueur de salle.

Structure de donnée

La première question à résoudre est de savoir comment on allait représenter le donjon en mémoire durant l'exécution du jeu.
Tout d'abord nous avons opté de séparer la structure de données en quatre niveaux (les portes, les templates de salles, les salles chargées et le donjon) pour simplifier l'utilisation. Les templates décrivent la configuration statique d'une salle, ses portes, son tag, sa fonction de construction, tandis que les salles chargées sont les instances effectivement présentes en mémoire avec leurs entités vivantes (murs, ennemis, objets, etc…).

Les portes et directions. Une porte est une entité à part entière qui hérite à la fois de block (elle a une position et une hitbox dans le monde) et de interact_resolver (le joueur peut interagir avec elle pour changer de salle). Elle possède un composant direction indiquant vers quelle direction elle mène :

```
class door () =
 let d = Component.init Up in
 object
   inherit block ()
   inherit interact_resolver ()
   method direction = d
 end
```

Les portes d'une salle sont regroupées dans un record de quatre door option, chaque champ pouvant être absent si la salle n'a pas de passage dans cette direction :

```
type doors = {
 up : door option; left : door option;
 down : door option; right : door option;
}
```

La salle. Une salle est également une entité du système ECS. Elle regroupe trois composants : ses murs (block array), ses portes (doors), et ses objets temporaires (ennemis, projectiles, items au sol) stockés dans une liste de deletable qui peuvent être supprimés à tout moment :

```
class room () =
 let walls = Component.init ([||] : block array) in
 let doors = Component.init ({up = None; left = None; down = None; right = None} : doors) in
 let temporary_objects = Component.init([] : deletable List.t) in
 object
   inherit Entity.t ()
   method walls = walls
   method doors = doors
   method temporary_objects = temporary_objects
 end
```

La carte du donjon. Pour représenter la topologie du donjon indépendamment des instances de salles, nous utilisons deux types différents. room_map décrit un blueprint de salle : il contient la configuration de portes, le tag de la salle (Boss_room, Start_room, Enemy_room, Treasure_room), et un room_creator, une fonction qui permet d'instancier la salle au moment où le joueur y entre, évitant de construire toutes les salles à l'avance.
La carte complète quand à elle est encodée dans un dungeon_layout :

```
type dungeon_layout = {
 rooms : room_map option array array;
 links : ((int * int) * (int * int)) list
}
```

Le champ rooms est une grille de blueprints : chaque cellule contient soit None si elle est inoccupée, soit une room_map décrivant la salle à cet emplacement. Le champ links est une liste d'arêtes entre coordonnées adjacentes qui encode les connexions entre salles. La grille et la liste de liens sont donc complémentaires, la grille dit quelles salles existent, la liste dit lesquelles sont connectées.

Le donjon. Enfin, la classe dungeon est l'entité racine qui centralise l'état global de l'exploration. Elle contient la carte (layout), la matrice des salles visitées (visited), la salle actuellement instanciée (current_room), les positions clés dans la grille (départ, boss, position courante), et le composant change_room qui sert de signal au système Room_loader pour déclencher un changement de salle :

```
class dungeon () =
 ...
 object
   inherit Entity.t ()
   method layout           = layout            (* dungeon_layout    *)
   method visited          = visited           (* bool array array  *)
   method current_room     = current           (* room option       *)
   method current_room_pos = current_room_pos  (* int * int         *)
   method start_room_pos   = start_room_pos    (* int * int         *)
   method boss_room_pos    = boss_room_pos     (* int * int         *)
   method change_room      = change_room       (* direction option  *)
end
```

Algorithme de génération en quatre étapes

Étape 1: Initialisation. La grille est créée entièrement à false. Une position de départ est tirée aléatoirement. La taille du chemin principal est calculée comme $N^2/5 - 3 + rand(7)$, ce qui garantit une densité raisonnable sans surcharger la grille.

Étape 2 : Chemin principal (main_path). Le chemin entre la salle de départ et la salle du boss est tracé par une exploration aléatoire de la grille. À chaque étape, on identifie les directions disponibles (voisins non encore visités et dans les bornes de la grille), on en tire une au hasard, et on avance. La fonction retourne la position finale (la salle du boss) et la liste des liens construits.

Étape 3 : Enrichissement du graphe. Une fois le chemin principal établi, deux passes viennent l'enrichir. La fonction add_annex_room initialise une file des salles pas encore traitées avec toutes les salles du chemin principal sauf pour celle du début et du boss pour qu'elle n'ai qu'un seul voisin. puis les traite une par une : pour chaque salle, on tente sur chacun de ses voisins vides d'y créer une salle, de se lier à elle et de l'ajouter à la fille des salles pas encore traitées avec une probabilité de 1/3. Les nouvelles salles ainsi créées rejoignent elles-mêmes la file, ce qui permet aux branches de se propager plusieurs fois. La fonction add_more_links ajoute ensuite, avec une probabilité de 1/4, des connexions supplémentaires entre salles déjà existantes et non encore reliées, créant des boucles et offrant au joueur plusieurs chemins possibles.

Étape 4 : Assignation des templates (assign_rooms). Pour chaque cellule active de la grille que l'on vient de générer, on calcule les caractéristiques que la salle doit avoir (les portes, le type de la salle, et on pourrait pour étendre le jeu rajouter d'autres conditions comme par exemple spécifier une zone ou un étage du donjon etc…). On va donc filtrer room_id_map (une bibliothèque de salle qui contient toutes les informations sur les salles que l'on a créé nous même à la main) pour ne garder que les templates qui vérifient les caractéristiques. Si plusieurs templates sont éligibles, l'un d'eux est choisi aléatoirement, assurant ainsi une variété même pour des configurations identiques.

Changement de salle

Le passage d'une salle à l'autre est géré par le système Room_loader, qui s'exécute à chaque frame et observe le composant change_room du donjon. Lorsque le joueur franchit une porte, ce composant est renseigné avec la direction empruntée. Le système décharge alors la salle courante, toutes ses entités (murs, ennemis, portes) sont marquées tokill pour être supprimées, puis instancie la nouvelle salle via son room_creator.
Cette fonction prend en paramètre un record visited_neighbours indiquant quelles salles adjacentes ont déjà été visitées, calculé au moment du changement de salle :

```
let doors = {
       up    = has_link links (i', j') (i' - 1, j') && visited.(i' - 1).(j');
       down  = has_link links (i', j') (i' + 1, j') && visited.(i' + 1).(j');
       left  = has_link links (i', j') (i', j' - 1) && visited.(i').(j' - 1);
```

```
        right = has_link links (i', j') (i', j' + 1) && visited.(i').(j' + 1);
} in
```

Ce record permet aux templates d'afficher les portes déjà empruntées avec une texture différente (allumée ou éteinte), donnant au joueur une information spatiale en plus de la carte. Enfin, le joueur est repositionné devant la porte d'entrée de la nouvelle salle grâce à get_door_pos, qui retrouve la porte opposée à la direction de transit : si le joueur entre par la droite, il est placé devant la porte gauche de la salle suivante.

Bilan

Cette architecture découpe proprement la topologie (la grille de cellules) de la géométrie (les templates de salles), ce qui permet d'étendre facilement la bibliothèque de salles sans modifier l'algorithme de génération. Ajouter une nouvelle salle se résume en effet à créer la salle et enregistrer un nouveau room_creator dans room_id_map avec le bon record de portes et le bon tag, le reste du système le prendra en compte automatiquement.
Plusieurs améliorations pourraient cependant être envisagées. La plus impactante pour le joueur serait la persistance des salles : actuellement, chaque fois que le joueur quitte une salle, elle est entièrement déchargée, et lorsqu'il y revient elle est reconstruite à neuf, les ennemis réapparaissent, les objets par terre disparaissent. Sauvegarder l'état des objets temporaires d'une salle avant de la décharger permettrait une exploration bien plus cohérente. Dans le même esprit, la bibliothèque de templates pourrait être enrichie : davantage de configurations différentes pour une même combinaison de portes rendraient chaque partie plus variée. Enfin, la formule de calcul de la taille du chemin principal et l'ajout de salle aléatoire introduit une variabilité qui peut générer des écarts importants entre les donjons, ce qui nuit à l'équilibre du jeu.

IV - Tests effectués

Approche générale

La nature du projet a rendu les tests unitaires difficiles à mettre en place : les différents composants du jeu sont fortement couplés entre eux, et il est rare qu'un morceau de code puisse être exercé de manière complètement isolée. Notre approche a donc été principalement empirique, tester en jouant au jeu et en cherchant à provoquer volontairement des situations susceptibles de produire des erreurs, comme par exemple poser deux armes exactement au même endroit pour vérifier qu'une seule des deux est ramassée.

Tests par affichage de debug

Un outil central dans notre démarche de test a été l'affichage de debug via Gfx.debug. La fonction print_dungeon en est un exemple représentatif : elle affiche dans la console une représentation ASCII du donjon généré, avec S pour la salle de départ, B pour la boss room, # pour les salles ordinaires, - pour les liens horizontaux et | pour les liens verticaux. Cela nous a

permis de vérifier visuellement que la topologie du donjon était correcte, que la minimap affichée en jeu correspondait bien à la réalité, et que les portes menaient bien aux bonnes salles. De manière similaire, nous avons utilisé des affichages ponctuels pour surveiller l'état des systèmes d'enregistrement d'entités en affichant le nombre d'éléments présents à chaque instant pour s'assurer qu'aucun objet n'était créé en double ou oublié lors d'un déchargement.

Tests fonctionnels ciblés

Nous avons également effectué des tests fonctionnels ciblés, en construisant des situations précises pour valider des fonctionnalités spécifiques. Les transitions de salles ont été testées dans les quatre directions pour vérifier que le joueur était toujours repositionné devant la bonne porte de la salle suivante. Nous avons généré un grand nombre de donjons à la suite pour nous assurer que le déplacement aléatoire pour créer le chemin principal ne se bloquait jamais prématurément et qu'un template correspondait toujours à chaque configuration de portes produite. Le comportement des projectiles a aussi été vérifié systématiquement (direction, nombre de balles créées, dégâts infligés) pour chaque type d'arme. Enfin, nous avons vérifié qu'après chaque changement de salle, toutes les entités de la salle précédente étaient bien déchargées et qu'aucune entité orpheline ne persistait en mémoire.
C'est lors des tests de collision que nous avons découvert un bug particulièrement subtil. Lorsque deux blocs étaient positionnés côte à côte on détectait quand même un collision sauf que le vecteur de pénétration calculé entre eux étaient donc nul. Or notre algorithme de résolution de collision normalise ce vecteur pour déterminer la direction de correction, ce qui provoque une division par zéro et introduit des valeurs NaN dans les positions des entités, corrompant l'état physique du jeu. Ce bug a été détecté après un test où nous avons créé une énorme quantité de blocs et après s’être rendu compte que certains blocs disparaissaient et que ces blocs avaient une vitesse égale à NaN.

Problèmes connus non corrigés

Nos tests nous ont également permis d'identifier plusieurs problèmes que nous avons choisi de ne pas corriger pour l'instant, car ils ne sont pas bloquants pour le joueur :

- **Taille de donjon minimale.** Si la taille du donjon demandée est extrêmement petite, la formule de calcul de la taille du chemin principal peut produire une valeur nulle. Ce cas ne se produit jamais dans des conditions normales d'utilisation.
- **Dispersion du fusil à pompe.** La dispersion présente un défaut de calcul : les balles partent toujours dans la bonne direction générale, mais selon l'angle visé, l'écart angulaire entre elles peut devenir nul et toutes les balles se superposent.
- **Delta time lors d'une mise en veille.** Lorsque le jeu passe en arrière-plan, des freeze peuvent avoir lieu, le delta time ainsi accumulé pendant le freeze peut être suffisamment grand pour que la gravité appliquée en un seul pas fasse traverser le sol au joueur.

V - Répartition du travail

Travail effectué ensemble

- Gestion des collisions.
- Affichage classique

Travail effectué par Matthieu Urios

- Gestion de la caméra
- Gestion du système de clear des entités
- Gestion des projectiles
- Gestion des armes pour pouvoir les utiliser, récupérer une arme au sol, etc…
- Gestion des ennemies et de leur ia
- Gestion des textures
- Gestion du système d'interaction

Travail effectué par Hector Bonhoure

- Gestion de la modification du système d’input
- Gestion du système de timer
- Gestion des déplacements du joueur
- Gestion du donjon, de ses salles et des portes
- Gestion des différents systèmes d’affichage (barre de vie, mini-map, menu de victoire et de mort, séparation entre background et affichage normal)
- Gestion du système click pour gérer les boutons